

计算理论导引第四次作业

by 易守拙 2024300001103

Problem 0.1.

证明:隐式对数空间可计算函数就是对数空间可计算函数.

Solution. 设 $f: \Sigma^* \rightarrow \Sigma^*$ 是隐式对数空间可计算的, 即存在对数空间图灵机 M 计算映射 $(x, i) \mapsto f(x)_i \wedge |f(x)| \leq \text{poly}(|x|)$ (这是因为 f 是 Implicitly logspace computable 的). 来构造一个对数空间图灵机 L 来计算 $f(x)$. L 的工作过程如下:

1. 初始化计数器 $i = 1$.
2. 循环直到 $i > |f(x)|$: a. 模拟 M 在输入 (x, i) 上的运行. b. 输出 M 的输出位 $f(x)_i$. c. $i = i + 1$.

由于 i 只需要 $O(\log|f(x)|) = O(\log|x|)$ 的空间, 且 M 也是对数空间的, 所以 L 总共只需要对数空间. 因此 f 是对数空间可计算的. $\square$

Problem 0.2.

我们证明了 QBF 可在线性空间判定. 将证明中描述的线性空间算法用程序实现.

Solution.

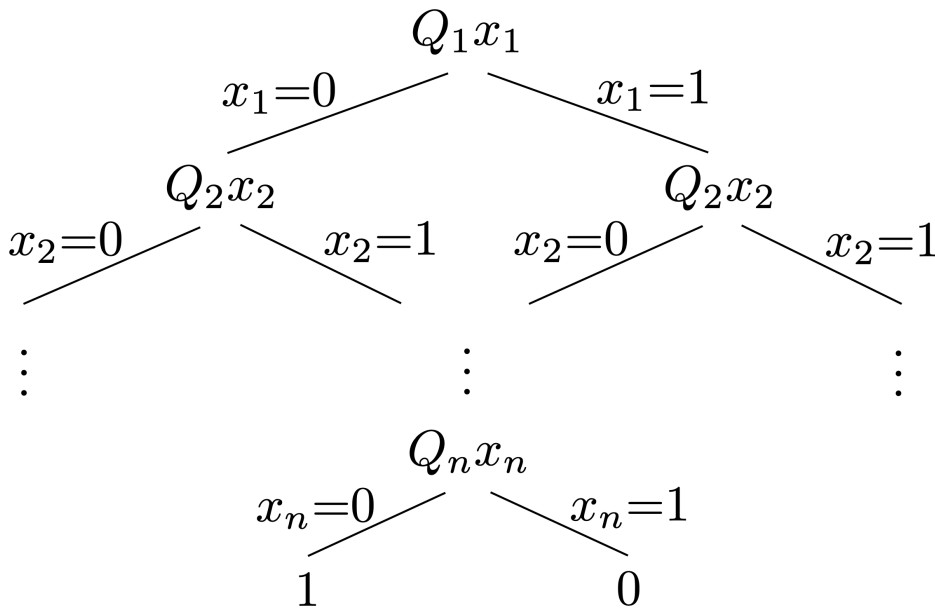


Figure 1: 课本上的图

1. 如果当前处理到无量词的情形, 直接根据赋值计算其真值.
2. 递归步骤:
 - 如果 $\varphi = \exists x\psi$, 依次尝试 $x = 0$ 和 $x = 1$.
 - 先设 $x = 0$, 递归调用 $\text{Eval}(\psi)$. 如果返回 True, 则 φ 为 True.
 - 如果 $x = 0$ 失败, 重用空间, 设 $x = 1$, 再次递归调用 $\text{Eval}(\psi)$.
 - 如果 $\varphi = \forall x\psi$, 类似地尝试 $x = 0$ 和 $x = 1$.
 - 只有当两次递归调用都返回 True 时, φ 才为 True.

伪代码:

```
def Eval(phi):
    if phi has no quantifiers:
        return Evaluate(phi, current_assignment)
```

```

Q, x, psi = Split(phi) # phi = Q x psi

# x = 0
current_assignment[x] = 0
val0 = Eval(psi)

if (Q == EXISTS and val0 == True): return True
if (Q == FORALL and val0 == False): return False

# x = 1
current_assignment[x] = 1
val1 = Eval(psi)

return val1

```

空间复杂度：每一层递归只需要存储当前变量 x 的值和少量控制信息。递归深度等于变量个数 m 。总空间 $O(m) = O(n)$ ，即线性空间。 $\square$

Problem 0.3.

说明斯托克迈尔-梅耶定理的证明中构造的 φ_x 是对数空间可计算的。

Solution. 回顾一下 $\varphi_x = \psi_x$ ，而 $\psi_i \triangleq$

$$(\exists C \forall D^1 \forall D^2 ((D^1 = C \wedge D^2 = F) \vee (D^1 = C \wedge D^2 = C')) \implies \psi_{i-1}(D^1, D^2)).$$

为了计算 ψ_i ，我们只需考虑这个图灵机 M ：它在工作带上存 C, D^1, D^2 ，并令维护一个计数器 ($O(\log(S(n)))$ 位)，用来记录目前算到哪个 ψ 了。

计算时，每次枚举 C, D^1, D^2 ，来计算蕴含的左边；如果左边为假，就继续枚举下一个；如果左边为真，递归的计算 ψ_{i-1} ，并更新计数器。由于 Problem 0.2，空间开销只在计数器那边，也就是它是对数空间开销的。进而是对数空间可计算的。 $\square$

Problem 0.4.

用程序实现萨维奇定理证明中的算法。

Solution. 我们通过“先左儿子，再右儿子，再祖先”的顺序 dfs 整个二叉树，利用枚举二进制编码来判断是否存在二叉树的那一个根。算法如下所示。

```

def Reach(u, v, k):
    # u: 起点, v: 终点, k: 二叉树深度
    if k == 0:
        return (u == v) or HasEdge(u, v)

    # 枚举中间点 w 的二进制编码
    for code from 0 to N-1:
        w = Decode(code)

        # left child
        if Reach(u, w, k-1):
            # rchild
            if Reach(w, v, k-1):
                return True

    return False

```

主程序调用：Reach(start_node, accept_node, ceil(log₂(N)))。

递归深度：log₂ N 。每层空间：存储节点 w ，需要 $O(\log N)$ 空间。总空间： $O(\log^2 N)$ 。由于 $N = 2^{S(n)}$ ， $O(\log^2 N) = O(S(n)^2)$ 。 $\square$

Problem 0.5.

证明: $A, B \in \mathbf{NP} \Rightarrow A \cap B, A \cup B, AB \in \mathbf{NP}$.

Solution. 设 $A, B \in \mathbf{NP}$. 存在非确定性多项式时间图灵机 N_A, N_B 分别判定 A, B .

1. $A \cap B$: 构造 $N_{\cap}$: 对于输入 w ,
 - a. 运行 $N_{A(w)}$. 如果 N_A 拒绝, 则拒绝.
 - b. 运行 $N_{B(w)}$. 如果 N_B 拒绝, 则拒绝.
 - c. 如果两者都接受, 则接受. 时间复杂度: $T_{A(n)} + T_{B(n)}$, 仍为多项式.
2. $A \cup B$: 构造 $N_{\cup}$: 对于输入 w ,
 - a. 非确定性地选择: 执行 (i) 或 (ii).
 - (i) 运行 $N_{A(w)}$, 输出其结果.
 - (ii) 运行 $N_{B(w)}$, 输出其结果.
 时间复杂度: $\max(T_{A(n)}, T_{B(n)})$, 仍为多项式.
3. AB : $AB = \{xy \mid x \in A, y \in B\}$. 构造 $N_{\{AB\}}$: 对于输入 w (长度 n),
 - a. 非确定性地选择分割点 $k \in \{0, \dots, n\}$.
 - b. 令 $x = w[1..k]$, $y = w[k+1..n]$.
 - c. 运行 $N_{A(x)}$. 如果拒绝, 则拒绝.
 - d. 运行 $N_{B(y)}$. 如果拒绝, 则拒绝.
 - e. 如果两者都接受, 则接受. 时间复杂度: $O(n) + T_{A(n)} + T_{B(n)}$, 仍为多项式.

因此 $A \cap B \in \mathbf{NP}$, $A \cup B \in \mathbf{NP}$, $AB \in \mathbf{NP}$.

□